

Vektor fazo modellari, embeddinglar va semantik munosabatlar

Tabiiy tilni qayta ishlash — 3-ma'ruza

A.A. Abdulali

11:00–12:20 · mavzu №3

18-iyun 2026

Prezentatsiya rejasi

- 1 Kirish: TF-IDF (Term Frequency–Inverse Document Frequency) dan embeddinggacha
- 2 Zich vektorlar: so'z embeddinglari
- 3 Kosinus o'xshashligi(cosine similarity)
- 4 So'z analogiyalari va semantik munosabatlar
- 5 PCA (Principal Component Analysis) bilan vizualizatsiya
- 6 Xulosa va amaliyotga ko'prik

Takrorlash: oldingi amaliyotda nima qurdik?

1-savol

2-amaliyotda sentiment klassifikatori **nima** ustida ishladi — xom matn ustidami yoki vektor ustida?

Vektorni qaysi usul yaratdi?

2-savol

TF-IDF fazosida "qiziq" va "qiziqarli" so'zlari bir-biriga **qanchalik yaqin**?

Takrorlash: oldingi amaliyotda nima qurdik?

1-savol

2-amaliyotda sentiment klassifikatori **nima** ustida ishladi — xom matn ustidami yoki vektor ustida?

Vektorni qaysi usul yaratdi?

2-savol

TF-IDF fazosida "qiziq" va "qiziqarli" so'zlari bir-biriga **qanchalik yaqin?**

1-javob

Vektor ustida — **TF-IDF / BoW** matritsasi. Model xom matnni emas, sonli vektorni ko'radi.

2-javob

Umuman yaqin emas! Ular ikki **alohida ustun** — o'xshashligi 0.

TF-IDF so'z ma'nosini bilmaydi. Bugun shuni hal qilamiz.

Bugungi dars maqsadlari

- 1 Distributiv gipoteza asosida so'z embeddingi nima uchun kerakligini tushuntirasiz.
- 2 TF-IDF vektorlarining semantik cheklovini *qiziq-qiziqarli* misolida izohlaysiz.
- 3 Ikki vektor orasidagi kosinus o'xshashligini qo'lda hisoblab, $\cos(a, b) = \frac{2}{3}$ natijasini talqin qilasiz.
- 4 Vektor arifmetikasidan foydalanib so'z analogiyasi uchun $t = \mathbf{b} - \mathbf{a} + \mathbf{c}$ maqsad vektorini tuzasiz.
- 5 `cosine_similarity`, `most_similar` funksiyalari va PCA natijasini matematik jihatdan izohlaysiz.

Minimum talablar: 1-ma'ruzadagi BoW va TF-IDF vektorlari; vektor, skalyar ko'paytma va Evklid normasi.

Bugungi darsda yechiladigan muammo

1-ma'ruzadan qolgan savol:

TF-IDF stemming yoki lemmatizatsiyasiz *qiziq* va *qiziqarli* so'zlarini ma'nan yaqin deb ko'ra oladimi?

Javob: yo'q

Lug'atda ular ikki alohida o'lcham sifatida turadi:

$qiziq \rightarrow (1, 0), \quad qiziqarli \rightarrow (0, 1)$

Kosinus o'xshashligi: $\cos = 0$.

Muammoning mohiyati:

- TF-IDF so'z shakllarini alohida belgi sifatida ko'radi.
- Semantik bog'liqlik bevosita kodlanmaydi.
- Sinonimlar ajralib qoladi: *avtomobil* $\neq$ *mashina*.

Zich vektor *embedding*: so'zni kichik o'lchamli vektor bilan ifodalaydi. Ma'nosi yaqin so'zlar fazoda yaqin joylashadi.

Prezentatsiya rejasi

- 1 Kirish: TF-IDF (Term Frequency–Inverse Document Frequency) dan embeddinggacha
- 2 Zich vektorlar: so'z embeddinglari
- 3 Kosinus o'xshashligi(cosine similarity)
- 4 So'z analogiyalari va semantik munosabatlar
- 5 PCA (Principal Component Analysis) bilan vizualizatsiya
- 6 Xulosa va amaliyotga ko'prik

So'zni raqamli vektorga qanday aylantiramiz?

Siyrak ifoda: one-hot / TF-IDF

$$nlp \rightarrow (0, \dots, 1, \dots, 0)$$

- Vektor uzunligi lug'at hajmiga teng: $|V|$.
- Ko'p komponentalari nol bo'ladi.
- Semantik yaqinlik bevosita ifodalanmaydi.

Zich ifoda: embedding

$$nlp \rightarrow (0.21, -0.08, 0.66, \dots)$$

- Odatda kichik o'lchamli: $d \ll |V|$.
- Qiymatlar matndan o'rganiladi.
- Ma'nosi yaqin so'zlar fazoda yaqinlashadi.

So'zning ma'nosi uning qanday kontekstlarda uchrashi orqali shakllanadi.

Misol: *choy* va *kofe* ko'pincha o'xshash kontekstlarda keladi.

<i>choy</i>	issiq	ichdim	finjon
<i>kofe</i>	issiq	ichdim	finjon

Xulosa

TF-IDF so'zning uchrashiga tayanadi, embedding esa uning kontekstini ham hisobga oladi.

So'z embeddingi: rasmiy ta'rif

Embedding lug'atdagi har bir so'zni kichik o'lchamli zich vektorga akslantiruvchi o'rganilgan funksiya:

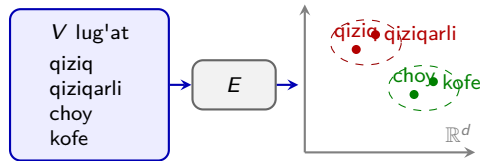
$$E : V \rightarrow \mathbb{R}^d, \quad d \ll |V|.$$

Belgilashlar

- V lug'at;
- $|V|$ lug'at hajmi;
- d embedding o'lchami;
- $E(w) = \mathbf{e}_w$ w so'zining vektori.

Taqqoslash

one-hot: $d = |V|$, embedding: $d \ll |V|$.



Asosiy g'oya

So'zlar alohida ustunlar emas, balki umumiy semantik fazodagi yaqin yoki uzoq nuqtalar sifatida qaraladi.

Embeddinglar qanday o'rganiladi? Uch asosiy yondashuv

Yondashuv	Asosiy g'oya	O'zbek tili uchun izoh
Word2Vec	So'z va uning atrofidagi kontekst orasidagi bog'liqlikni o'rganadi. CBOW va Skip-gram usullari ishlatiladi.	Korpus yetarli bo'lsa, tez va amaliy.
GloVe	So'zlarning butun korpus bo'yicha birga uchrashish statistikasidan foydalanadi.	Katta va toza korpus talab qiladi.
FastText	So'zni belgilar n-gramlari yig'indisi sifatida ifodalaydi.	Agglutinativ tillar va yangi so'zlar uchun foydali.

Bu ma'ruzada e'tibor nimaga qaratiladi?

Embeddinglarni o'qitish tafsilotlariga hozircha chuqur kirmaymiz. Asosiy maqsad tayyor vektorlar orqali o'xshashlikni hisoblash, so'z analogiyalarini tushuntirish va PCA yordamida ularni vizual ko'rsatish.

Nega one-hot vektorlar ma'noni to'liq ifodalay olmaydi?

1-qadam. Ikki har xil so'zning one-hot vektorlari:

$$qiziq \rightarrow \mathbf{u} = (1, 0, 0, \dots), \quad qiziqarli \rightarrow \mathbf{v} = (0, 1, 0, \dots)$$

2-qadam. Skalyar ko'paytmani hisoblaymiz:

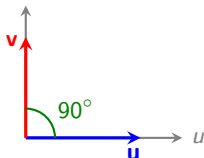
$$\mathbf{u} \cdot \mathbf{v} = 1 \cdot 0 + 0 \cdot 1 + \dots = 0.$$

3-qadam. Demak, one-hot fazosida ikki har xil so'zning o'xshashligi odatda 0 chiqadi ma'nosi yaqin bo'lsa ham.

Xulosa: one-hot ifoda so'zlarni alohida ustunlar sifatida ko'radi. Shu sababli semantik yaqinlik bevosita ifodalanmaydi. Embedding esa ma'nosi yaqin so'zlarni fazoda bir-biriga yaqinlashtirishga xizmat qiladi.

Qo'lda misol: bir xil so'zlar, ikki xil ifoda

One-hot: siyrak ifoda



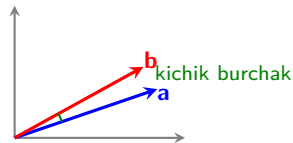
$$qiziq \rightarrow \mathbf{u} = (1, 0)$$

$$qiziqarli \rightarrow \mathbf{v} = (0, 1)$$

$$\mathbf{u} \cdot \mathbf{v} = 0, \quad \cos(\mathbf{u}, \mathbf{v}) = 0$$

Natija: ma'nosi yaqin bo'lsa ham, one-hot fazosida ular yaqin emas.

Embedding: zich ifoda



$$qiziq \rightarrow \mathbf{a} = (0.9, 0.4)$$

$$qiziqarli \rightarrow \mathbf{b} = (0.8, 0.5)$$

$$\mathbf{a} \cdot \mathbf{b} = 0.9 \cdot 0.8 + 0.4 \cdot 0.5 = 0.92$$

Natija: sifatli embeddingda ma'nosi yaqin so'zlar fazoda ham yaqin joylashadi.

Sizning vazifangiz: qaysi juft yaqinroq?

Vazifa

Distributiv gipotezaga ko'ra, yaxshi o'qitilgan embedding fazosida qaysi juft so'z bir-biriga yaqinroq bo'lishi kerak?

(a) (*avtomobil, mashina*) (b) (*avtomobil, banan*)

To'g'ri javob:

(*avtomobil, mashina*)

Chunki bu ikki so'z ko'pincha o'xshash kontekstlarda uchraydi: *haydadi, sotib oldi, ta'mirladi*.

Kontekstlar orqali asoslash:

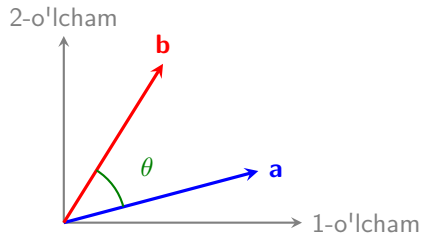
<i>avtomobil</i>	haydadi, sotib oldi, ta'mirladi
<i>mashina</i>	haydadi, sotib oldi, ta'mirladi
<i>banan</i>	yedi, meva, sariq

Xulosa: kontekstlari o'xshash so'zlarning embedding vektorlari odatda yaqin joylashadi.

Prezentatsiya rejasi

- 1 Kirish: TF-IDF (Term Frequency–Inverse Document Frequency) dan embeddinggacha
- 2 Zich vektorlar: so'z embeddinglari
- 3 Kosinus o'xshashligi(cosine similarity)**
- 4 So'z analogiyalari va semantik munosabatlar
- 5 PCA (Principal Component Analysis) bilan vizualizatsiya
- 6 Xulosa va amaliyotga ko'prik

Intuitsiya: o'xshashlik vektorlar orasidagi burchak



Kosinus o'xshashligi vektorlarning uzunligini emas, ular orasidagi **yo'nalish yaqinligini** baholaydi.

Asosiy g'oya

- Har bir so'z yoki hujjat fazodagi vektor sifatida qaraladi.
- Burchak kichik bo'lsa, vektorlar o'xshash yo'nalishda.
- Burchak 90° ga yaqin bo'lsa, bog'liqlik kuchsiz.

Nega uzunlik emas?

Bir mavzudagi qisqa va uzun matnlarning vektor uzunligi farq qilishi mumkin. Kosinus esa ularning yo'nalishini solishtiradi.

Kosinus o'xshashligi: rasmiy ta'rif

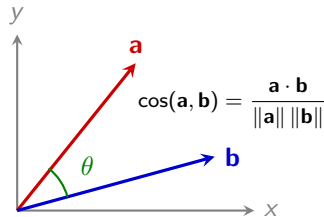
$$\cos(\mathbf{a}, \mathbf{b}) = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\| \|\mathbf{b}\|} = \frac{\sum_{i=1}^n a_i b_i}{\sqrt{\sum_{i=1}^n a_i^2} \sqrt{\sum_{i=1}^n b_i^2}}, \quad \mathbf{a}, \mathbf{b} \neq \mathbf{0}.$$

Belgilashlar

- $\mathbf{a}, \mathbf{b} \in \mathbb{R}^n$ TF-IDF yoki embedding vektorlar;
- $\mathbf{a} \cdot \mathbf{b}$ skalyar ko'paytma;
- $\|\mathbf{a}\|, \|\mathbf{b}\|$ Evklid normalari;
- n vektor o'lchami.

Talqin

- $\cos = 1$ bir xil yo'nalish;
- $\cos = 0$ ortogonal, umumiy yo'nalish yo'lq;



Burchak kichraygan sari kosinus qiymati kattalashadi; burchak kattalashgan sari o'xshashlik kamayadi.

Keltirib chiqarish: nega normaga bo'lamiz?

1-qadam. Skalyar ko'paytmaning geometrik ta'rifi:

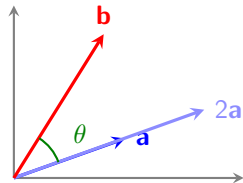
$$\mathbf{a} \cdot \mathbf{b} = \|\mathbf{a}\| \|\mathbf{b}\| \cos \theta.$$

2-qadam. Bizga kerak bo'lgan $\cos \theta$ ni ajratamiz:

$$\cos \theta = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\| \|\mathbf{b}\|}.$$

3-qadam. Agar vektorni ikki marta uzaytirsak, yo'nalish o'zgarmaydi:

$$\cos(2\mathbf{a}, \mathbf{b}) = \frac{2(\mathbf{a} \cdot \mathbf{b})}{2\|\mathbf{a}\| \|\mathbf{b}\|} = \cos(\mathbf{a}, \mathbf{b}).$$



Xulosa: kosinus o'xshashligi uzunlikni emas, vektorlarning yo'nalishini solishtiradi. Shuning uchun u matn uzunligi turlicha bo'lganda ham qulay.

Qo'lda hisob: ikki vektorning kosinus o'xshashligi

$$H_1 : \text{"nlp juda qiziq"} \rightarrow \mathbf{a} = (1, 1, 1, 0), \quad H_2 : \text{"nlp juda foydali"} \rightarrow \mathbf{b} = (1, 1, 0, 1)$$

Lug'at tartibi:

(nlp, juda, qiziq, foydali)

Hisoblash

$$\mathbf{a} \cdot \mathbf{b} = 1 \cdot 1 + 1 \cdot 1 + 1 \cdot 0 + 0 \cdot 1 = 2$$

$$\|\mathbf{a}\| = \sqrt{1^2 + 1^2 + 1^2} = \sqrt{3}, \quad \|\mathbf{b}\| = \sqrt{3}$$

$$\cos(\mathbf{a}, \mathbf{b}) = \frac{2}{\sqrt{3}\sqrt{3}} = \frac{2}{3} \approx 0.667$$

Talqin

H_1 va H_2 matnlarida ikki so'z umumiy:

nlp, juda

Shuning uchun kosinus o'xshashligi noldan katta.

Eslatma: juda kabi umumiy so'zlar amaliy NLP tizimlarida stop-word sifatida olib tashlanishi mumkin.

Sizning vazifangiz: kosinusni hisoblang

Vazifa

H_1 : “nlp juda qiziq” $\rightarrow \mathbf{a} = (1, 1, 1, 0, 0, 0)$

H_3 : “python juda qulay” $\rightarrow \mathbf{c} = (0, 1, 0, 0, 1, 1)$

Lug'at tartibi:

(*nlp, juda, qiziq, foydali, python, qulay*)

$$\cos(\mathbf{a}, \mathbf{c}) = ?$$

Yechim

$$\mathbf{a} \cdot \mathbf{c} = 1 \cdot 0 + 1 \cdot 1 + 1 \cdot 0 + 0 \cdot 0 + 0 \cdot 1 + 0 \cdot 1 = 1$$

$$\|\mathbf{a}\| = \sqrt{3}, \quad \|\mathbf{c}\| = \sqrt{3}$$

$$\cos(\mathbf{a}, \mathbf{c}) = \frac{1}{\sqrt{3}\sqrt{3}} = \frac{1}{3} \approx 0.333$$

Talqin: ikki gap faqat *juda* so'zida mos keladi. Asosiy mavzu boshqa: *nlp* va *python*. Shuning uchun o'xshashlik past.

Kod va formula: kosinus qanday hisoblanadi?

Python kodi

```
import numpy as np
from sklearn.metrics.pairwise import (
    cosine_similarity
)

a = np.array([[1, 1, 1, 0]])
b = np.array([[1, 1, 0, 1]])

dot = (a * b).sum()
na = np.linalg.norm(a)
nb = np.linalg.norm(b)

cos_val = dot / (na * nb)
print(cos_val) # 0.667
print(cosine_similarity(a, b)) # [[0.667]]
```

Formula bilan moslik

Kod	Formula
$(a*b).sum()$	$\sum_i a_i b_i$
$norm(a)$	$\ a\ $
$norm(b)$	$\ b\ $
$dot/(na*nb)$	$\cos(a, b)$

`cosine_similarity` funksiyasi kosinus formulasini tayyor holda hisoblaydi:

$$\cos(a, b) = \frac{a \cdot b}{\|a\| \|b\|}.$$

3-amaliyotda shu mantiq haqiqiy embedding vektorlariga qollanadi.

Kosinus tuzog'i: 1D emas, 2D massiv kerak

Tez-tez uchraydigan xato

cosine_similarity kirishda 2D massiv kutadi:

$(n_samples, n_features)$

1D vektor berilsa:

Expected 2D array

Qoida: embedding odatda (100,) shaklida keladi; uni bitta namuna sifatida berish uchun (1,100) shakliga otkazamiz.

Minimal tuzatish

```
a = np.array([1, 1, 1, 0])
b = np.array([1, 1, 0, 1])

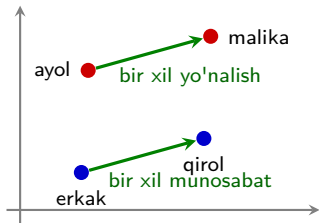
# Xato:
cosine_similarity(a, b)

# To'g'ri:
cosine_similarity(
    a.reshape(1, -1),
    b.reshape(1, -1)
)
```

Prezentatsiya rejasi

- 1 Kirish: TF-IDF (Term Frequency–Inverse Document Frequency) dan embeddinggacha
- 2 Zich vektorlar: so'z embeddinglari
- 3 Kosinus o'xshashligi(cosine similarity)
- 4 So'z analogiyalari va semantik munosabatlar
- 5 PCA (Principal Component Analysis) bilan vizualizatsiya
- 6 Xulosa va amaliyotga ko'prik

Intuitsiya: semantik munosabat vektor yo'nalishi sifatida



Asosiy g'oya

- Embedding fazosida ayrim semantik munosabatlar vektorlar orasidagi yo'nalish orqali namoyon bo'ladi.

- Masalan:

$$E(qirol) - E(erkak) \approx E(malika) - E(ayol)$$

- Shu sababli so'z analogiyalarini vektor arifmetikasi orqali izohlash mumkin:

$$king - man + woman \approx queen.$$

Bu natija faqat sifatli va yetarli korpusda o'qitilgan embeddinglarda aniqroq ko'rinadi.

So'z analogiyasi: rasmiy ta'rif

Analogiya savoli

$$a : b :: c : ?$$

Mazmuni: a ga b qanday munosabatda bo'lsa, c ga qaysi so'z shunday munosabatda?

$$\mathbf{t} = \mathbf{b} - \mathbf{a} + \mathbf{c}$$

$\mathbf{t}$ javob so'z emas, balki qidiriladigan so'zga mos maqsad vektor.

$$\hat{d} = \arg \max_{w \in V \setminus \{a, b, c\}} \cos(E(w), \mathbf{t})$$

Formula qanday o'qiladi?

- $\mathbf{b} - \mathbf{a}$ a dan b ga semantik siljish.
- Shu siljish $\mathbf{c}$ ga qo'shiladi.
- $\mathbf{t}$ ga eng yaqin so'z kosinus orqali topiladi.

Masalan:

$$\textit{king} - \textit{man} + \textit{woman} \approx \textit{queen}.$$

Amaliy kutubxonalarda bu g'oya ko'pincha `most_similar` funksiyasi orqali bajariladi.

Analogiyani 2D vektorlarda qo'lda yechish

Analogiya savoli

O'zbekiston : Toshkent :: Rossiya : ?

Izoh: sonlar haqiqiy embedding emas, mexanizmni tushuntirish uchun berilgan 2D misoldir.

$$E(O'zbekiston) = (2, 0), \quad E(Toshkent) = (3, 1)$$

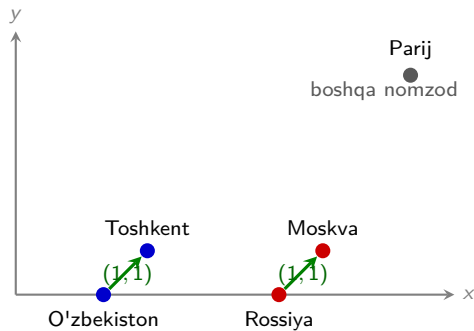
$$E(Rossiya) = (6, 0), \quad E(Moskva) = (7, 1), \quad E(Parij) = (9, 5)$$

1-qadam. Semantik siljishni topamiz

$$\Delta = E(Toshkent) - E(O'zbekiston) = (3, 1) - (2, 0) = (1, 1)$$

2-qadam. Siljishni Rossiyaga qo'llaymiz

$$t = E(Rossiya) + \Delta = (6, 0) + (1, 1) = (7, 1) = E(Moskva)$$



Bir xil semantik siljish qo'llanadi:

O'zbekiston → *Toshkent*, *Rossiya* → *Moskva*.

Sizning vazifangiz: analogiyani yeching

Vazifa

erkak : qirol :: ayol : ?

$$E(\text{erkak}) = (1, 1), \quad E(\text{qirol}) = (3, 2)$$

$$E(\text{ayol}) = (1, 3)$$

$$E(\text{malika}) = (3, 4), \quad E(\text{dehqon}) = (2, 1)$$

Yechim

$$\mathbf{t} = \mathbf{b} - \mathbf{a} + \mathbf{c} = (3, 2) - (1, 1) + (1, 3) = (3, 4)$$

$$\mathbf{t} = (3, 4) = E(\text{malika})$$

Javob: *malika*. Chunki $\mathbf{t}$ vektori *malika* vektori bilan aynan mos keladi; *dehqon* esa uzoqroq nomzod.

Kod va formula: most_similar orqali analogiya

Gensim kodi

```
from gensim.models import KeyedVectors

wv = KeyedVectors.load("cc_uz_100k.kv")

# Eng yaqin so'zlar
wv.most_similar("toshkent", topn=3)

# Analogiya:
# O'zbekiston : Toshkent :: Rossiya : ?
wv.most_similar(
    positive=["toshkent", "rossiya"],
    negative=["uzbekiston"],
    topn=1
)

# Natija: [("moskva", 0.78)]
```

Formula bilan bog'lanishi

$$t = E(\textit{Toshkent}) - E(\textit{O'zbekiston}) + E(\textit{Rossiya})$$

Kod	Formula
positive=[b,c]	$+\mathbf{b} + \mathbf{c}$
negative=[a]	$-\mathbf{a}$
most_similar	arg max cos

Muhim: positive ro'yxatidagi vektorlar qo'shiladi, negative ro'yxatidagilar ayiriladi. Natija maqsad vektorga eng yaqin so'z sifatida topiladi.

3-amaliyotda aynan shu API ishlatiladi.

Prezentatsiya rejasi

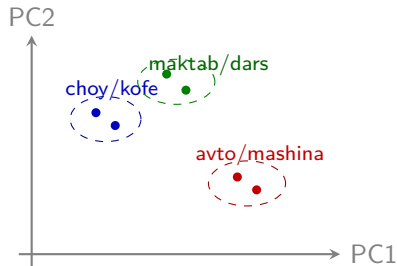
- 1 Kirish: TF-IDF (Term Frequency–Inverse Document Frequency) dan embeddinggacha
- 2 Zich vektorlar: so'z embeddinglari
- 3 Kosinus o'xshashligi(cosine similarity)
- 4 So'z analogiyalari va semantik munosabatlar
- 5 PCA (Principal Component Analysis) bilan vizualizatsiya**
- 6 Xulosa va amaliyotga ko'prik

300 o'lchamli embeddingni qanday ko'ramiz?

Muammo

- Embeddinglar odatda 100–300 o'lchamli bo'ladi.
- Inson esa 2D yoki 3D fazoni qulay ko'ra oladi.
- Shuning uchun yuqori o'lchamli vektorlarni 2D ga tushirish kerak.

PCA (*Principal Component Analysis*) yuqori o'lchamli nuqtalarni eng ko'p dispersiya saqlanadigan 2 ta yo'nalishga proyeksiya qiladi.



Ehtiyot bo'ling: PCA chizmasi taxminiy ko'rinish beradi. Aniq yaqinlikni baholash uchun asl embedding fazosida kosinus hisoblanadi.

PCA: asosiy komponentlar tahlili

PCA (*Principal Component Analysis*) yuqori o'lchamli ma'lumotni dispersiya eng ko'p saqlanadigan yangi o'qlarga proyeksiya qiluvchi o'lcham qisqartirish usuli.

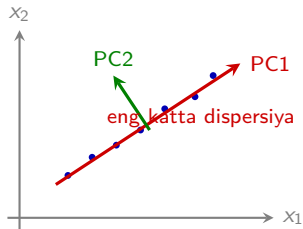
Matematik g'oya

$$X_c = X - \bar{x}$$

$$C = \frac{1}{n-1} X_c^T X_c$$

$$C \mathbf{v}_i = \lambda_i \mathbf{v}_i, \quad \lambda_1 \geq \lambda_2 \geq \dots$$

- X_c markazlashtirilgan ma'lumot;
- C kovariatsiya matritsasi;
- $\mathbf{v}_i$ asosiy komponent yo'nalishi;
- λ_i shu yo'nalishdagi dispersiya.



PC1 dispersiya eng katta bo'lgan yo'nalish. PC2 PC1 ga perpendikular keyingi asosiy yo'nalish.

Qo'lda misol: qaysi o'q ko'proq ma'lumot saqlaydi?

Uchta markazlashtirilgan nuqta

$$w_1 = (2, 0.1), \quad w_2 = (-2, -0.1), \quad w_3 = (0, 0)$$

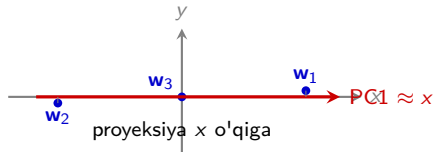
Har bir o'q bo'yicha tarqalish

$$\text{Var}(x) \propto 2^2 + (-2)^2 + 0^2 = 8$$

$$\text{Var}(y) \propto 0.1^2 + (-0.1)^2 + 0^2 = 0.02$$

$$\frac{\text{Var}(x)}{\text{Var}(y)} = \frac{8}{0.02} = 400$$

Demak, nuqtalar asosan x o'qi bo'ylab tarqalgan. PCA birinchi komponent sifatida shu yo'nalishni saqlaydi.



$$\frac{8}{8 + 0.02} \approx 0.998$$

Bitta komponent deyarli 99.8% dispersiyani saqlaydi.

PCA nimani saqlaydi, nimani yo'qotadi?

Vazifa

300 o'lchamli o'zbek embeddinglari PCA orqali 2D fazoga tushirildi:

```
explained_variance_ratio_ = [0.31, 0.18]
```

Savollar:

- 1 2D tasvir ma'lumotning qancha qismini saqlaydi?
- 2 2D da yaqin ko'ringan so'zlar asl fazoda ham yaqin bo'lishi shartmi?

Yechim

$$0.31 + 0.18 = 0.49 \Rightarrow 49\%$$

$$1 - 0.49 = 0.51 \Rightarrow 51\%$$

- 2D proyeksiya dispersiyaning **49%** qismini saqlaydi.
- Qolgan **51%** dispersiya 2D tasvirda aks etmaydi.
- Shuning uchun 2D yaqinlik faqat taxminiy signal.

Aniq semantik yaqinlik asl embedding fazosida kosinus o'xshashligi orqali baholanadi.

Kod va formula: PCA bilan 2D vizualizatsiya

Python kodi

```
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt

# X: (N, d) N ta so'z, d o'lchamli embedding
pca = PCA(n_components=2)

Z = pca.fit_transform(X) # Z: (N, 2)

plt.scatter(Z[:, 0], Z[:, 1])

for i, word in enumerate(words):
    plt.annotate(word, Z[i])

print(pca.explained_variance_ratio_)
```

Formula bilan bog'lanishi

$$X_c = X - \bar{x}$$

$$Z = X_c \begin{bmatrix} \mathbf{v}_1 & \mathbf{v}_2 \end{bmatrix}$$

Kod	Ma'nosi
n_components=2	PC1, PC2
fit_transform(X)	2D proyeksiya
Z[:,0]	PC1 koordinata
Z[:,1]	PC2 koordinata
annotate	so'z yorlig'i

explained_variance_ratio_ PC1 va PC2 qancha dispersiya saqlaganini ko'rsatadi.

Eslatma: PCA chizmasi taxminiy talqin beradi; aniq yaqinlik asl embedding fazosida kosinus orqali baholanadi.

Prezentatsiya rejasi

- 1 Kirish: TF-IDF (Term Frequency–Inverse Document Frequency) dan embeddinggacha
- 2 Zich vektorlar: so'z embeddinglari
- 3 Kosinus o'xshashligi(cosine similarity)
- 4 So'z analogiyalari va semantik munosabatlar
- 5 PCA (Principal Component Analysis) bilan vizualizatsiya
- 6 Xulosa va amaliyotga ko'prik

O'zbek tili uchun embedding: qamrov, agglutinatsiya va subword

Asosiy muammolar

- **Qamrov:** model lug'atida bo'lmagan so'zlar OOV muammosini keltirib chiqaradi.
- **Agglutinatsiya:** bitta ildizdan ko'plab shakllar hosil bo'ladi:

kitob, kitoblar, kitoblarimizdan.

- **Kam resurslilik:** katta, toza va balanslangan o'zbek korpuslari yetarli bo'lmasa, embedding sifati pasayadi.

Bir xil ma'noga yaqin shakllar alohida token sifatida ko'rilsa, ular orasidagi semantik bog'liqlik zaiflashadi.

Amaliy yechimlar

- **Normalizatsiya:** imlo, apostrof, lotin-kiril yozuvi va tokenizatsiyani bir xil standartga keltirish.
- **Lemmatizatsiya/stemming:** so'z shakllarini umumiy ildiz yoki lemma atrofida birlashtirish.
- **Subword yondashuv:** FastText yoki BPE/WordPiece orqali so'zni kichik birliklarga ajratish.

Subword yondashuv OOV muammosini kamaytiradi: model butun so'zni bilmasa ham, uning qismlaridan vektor hosil qilishi mumkin.

kitoblarimizdan $\approx$ kitob + lar + imiz + dan

Qachon nima ishlatamiz? Siyrak va zich vektorlar taqqoslovi

Mezon	Siyrak vektorlar: BoW / TF-IDF	Zich vektorlar: embedding
Ifoda	Juda katta o'lchamli, ko'p komponentalari nol.	Kichik o'lchamli, komponentalari zich.
Ma'no	So'z chastotasi va hujjatdagi ahamiyatini aks ettiradi.	Kontekst va semantik yaqinlikni ifodalashga harakat qiladi.
Afzallik	Sodda, tez, talqin qilish oson. Kichik datasetlarda yaxshi ishlaydi.	Sinonimlik, yaqin ma'no va analogiyalarni yaxshiroq ushlaydi.
Kamchilik	Sinonimlarni va yashirin semantik bog'liqlikni ko'rmaydi.	Ko'proq korpus, sifatli model va ehtiyotkor talqin talab qiladi.
Qachon?	Klassik tasniflash, qidiruv, baseline model.	Semantik qidiruv, klasterlash, analogiya, transformer modellar.

Amaliy tavsiya: avval TF-IDF bilan kuchli baseline quriladi; semantik yaqinlik muhim bo'lsa, embedding yoki transformer embeddinglariga o'tiladi.

Seminal maqola: GloVe (Pennington va b., 2014)

Asosiy g'oya

- GloVe *Global Vectors for Word Representation*.
- Model butun korpus bo'yicha so'zlarning birga uchrashish statistikasi asosida o'rganadi.
- Faqat lokal kontekst emas, global *co-occurrence* matritsasi ham hisobga olinadi.

GloVe g'oyasi: so'zlar orasidagi ma'no farqi ularning turli kontekst so'zlari bilan uchrashish nisbatlarida ko'rinadi.

Model formulasi

$$J = \sum_{i,j=1}^{|V|} f(X_{ij}) (\mathbf{w}_i^\top \tilde{\mathbf{w}}_j + b_i + \tilde{b}_j - \log X_{ij})^2$$

- X_{ij} i va j so'zlarining birga uchrashish soni;
- $\mathbf{w}_i, \tilde{\mathbf{w}}_j$ o'rganiladigan vektorlar;
- $f(X_{ij})$ juda kam yoki juda ko'p uchrashlarni muvozanatlaydi.

Amaliy talqin: GloVe katta va toza korpusda yaxshi ishlaydi; kichik yoki shovqinli korpusda natija barqaror bo'lmasligi mumkin.

Bugungi maqsadlar bajarildi

Bugun nimalarni tushundik?

- TF-IDF semantik yaqinlikni har doim ham ifodalay olmasligini ko'rdik.
- So'z embeddingi $E : V \rightarrow \mathbb{R}^d$ sifatida ta'riflandi.
- Kosinus o'xshashligi orqali vektorlar yo'nalishi taqqoslandi.
- So'z analogiyasi $t = b - a + c$ orqali tushuntirildi.

Amaliy bog'lanish

- `cosine_similarity` vektorlar o'xshashligini hisoblaydi.
- `most_similar` maqsad vektorga eng yaqin so'zni topadi.
- PCA embeddinglarni 2D fazoda ko'rishga yordam beradi.

Muhim xulosa: PCA faqat vizual talqin beradi; aniq semantik yaqinlik asl embedding fazosida kosinus orqali baholanadi.

3-amaliyot: tayyor embeddinglar bilan ishlash

Amaliyotda bajariladigan ishlar

- 1 Tayyor embedding modelini yuklash.
- 2 So'z vektorini olish va uning o'lchamini tekshirish.
- 3 Ikki so'z orasidagi kosinus o'xshashligini hisoblash.
- 4 `most_similar` orqali eng yaqin so'zlarni topish.
- 5 PCA yordamida tanlangan so'zlarni 2D fazoda ko'rsatish.

Asosiy e'tibor modelni qayta o'qitishga emas, tayyor embedding vektorlari bilan to'g'ri ishlashga qaratiladi.

Kutiladigan natijalar

- *choy–kofe, avtomobil–mashina* kabi juftliklarda yaqinlikni tekshirasiz.
- OOV so'zlar va apostrof/imlo farqlari model natijasiga qanday ta'sir qilishini ko'rasiz.
- PCA chizmasida semantik guruhlar qanday ko'rinishini tahlil qilasiz.

Ehtiyot bo'ling: 2D PCA chizmasi faqat vizual yordam. Yakuniy xulosa asl embedding fazosidagi kosinus o'xshashligi asosida beriladi.

- ① Pennington, J., Socher, R., Manning, C.D. (2014). *GloVe: Global Vectors for Word Representation*. EMNLP 2014.
- ② Mikolov, T. va b. (2013). *Efficient Estimation of Word Representations in Vector Space*. ICLR 2013. (Word2Vec — L6 da batafsil.)
- ③ Jurafsky, D., Martin, J.H. (2023). *Speech and Language Processing*, 3-nashr (online). 6-bob: Vector Semantics and Embeddings.
- ④ Firth, J.R. (1957). *A Synopsis of Linguistic Theory*. (Distributiv gipoteza manbasi.)
- ⑤ Scikit-learn hujjatlari: `cosine_similarity`, PCA — scikit-learn.org.

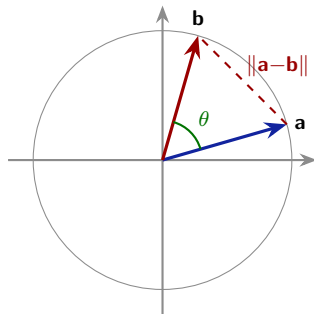
Ilova S1: kosinus va Evklid masofa bog'liqligi

L_2 -normallashtirilgan vektorlar uchun ($\|\mathbf{a}\| = \|\mathbf{b}\| = 1$):

$$\|\mathbf{a} - \mathbf{b}\|^2 = \|\mathbf{a}\|^2 + \|\mathbf{b}\|^2 - 2\mathbf{a} \cdot \mathbf{b} = 2 - 2\cos(\mathbf{a}, \mathbf{b})$$

bunda: $\|\mathbf{a}\| = \|\mathbf{b}\| = 1$ deb olindi.

Demak normallashtirilgan fazoda Evklid masofasi va kosinus **ekivalent tartib** beradi — shuning uchun ko'p kutubxonalar ikkalasini almashtirib ishlatadi. Bu LSH va qidiruvda muhim (L4, P4).



Birlik aylanada: vatar = $\|\mathbf{a} - \mathbf{b}\|$, burchak θ

Ilova S2: vektor, norma va skalyar ko'paytma eslatmasi

Skalyar ko'paytma: $\mathbf{a} \cdot \mathbf{b} = \sum_i a_i b_i$

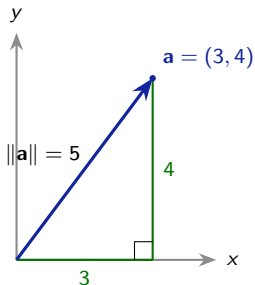
Evklid normasi (uzunlik): $\|\mathbf{a}\| = \sqrt{\sum_i a_i^2}$

Perpendikular vektorlar: $\mathbf{a} \cdot \mathbf{b} = 0$

Misol: $\mathbf{a} = (3, 4)$ uchun $\|\mathbf{a}\| = \sqrt{9 + 16} = \sqrt{25} = 5$.

Python da: `np.linalg.norm([3,4]) → 5.0;`

`np.dot(a, b)` — skalyar ko'paytma.



Norma = gipotenuza = $\sqrt{3^2 + 4^2} = 5$

Ilova S3: PCA matematik asoslari

- 1 Ma'lumotni markazlashtiring: har ustundan o'rtachani ayiring.
- 2 Kovariatsiya matritsasi: $C = \frac{1}{n}X^T X$.
- 3 C ning xos qiymat va xos vektorlarini toping.
- 4 Eng katta k xos qiymatga mos xos vektorlarni tanlang — bular asosiy komponentlar.
- 5 Ma'lumotni shu k vektorga proyeksiya qiling.

`explained_variance_ratio_[i]` = $\lambda_i / \sum_j \lambda_j$, bunda λ_i — i -xos qiymat. Bu kursda PCA ni `sklearn` amalga oshiradi — qo'lda xos qiymat hisoblanmaydi.